

PUC CAMPINAS

Pontifícia Universidade Católica de Campinas
Escola Politécnica

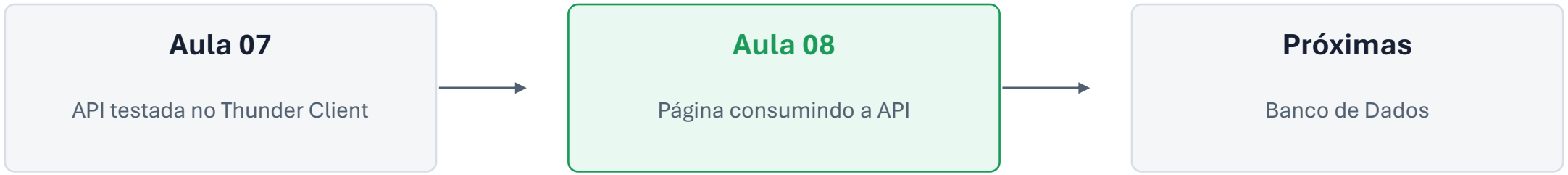
Frontend consumindo API REST com fetch

INTRODUÇÃO AS TECNOLOGIAS WEB

José Guilherme Picolo
E-mail: jose.picolo@puc-campinas.edu.br

Onde esta aula se encaixa?

A Aula 08 fecha o primeiro ciclo completo de integração frontend–backend.



Antes desta aula

A API já existe e responde em JSON, mas só foi chamada manualmente em ferramentas de teste. (Thunder Client)

Depois desta aula

O JavaScript do navegador passa a chamar a API e atualizar o HTML dinamicamente.

Backend reaproveitado da Aula 07

A aula parte da API de alunos já implementada no Express.

Rotas disponíveis

Método	Rota	Finalidade	Dados
GET	/alunos	listar alunos ou filtrar por situação	req.query
GET	/alunos/:ra	buscar um aluno específico	req.params
POST	/alunos	cadastrar novo aluno	req.body
PUT	/alunos/:ra	atualizar aluno existente	params + body
DELETE	/alunos/:ra	inativar aluno	req.params

Ponto didático

Nesta aula, o backend continuará usando array em memória. O objetivo agora é substituir o teste manual por chamadas feitas pelo frontend.

Arquivos do frontend

As telas já vêm prontas; o foco da aula será completar as funções JavaScript.

```
frontend/
├── index.html
├── pages/
│   ├── listar-alunos.html
│   └── manipular-aluno.html
├── css/
│   └── style.css
└── js/
    ├── api.js
    ├── alunos-lista.js
    └── aluno-formulario.js
```

HTML + Bootstrap

Estrutura visual das páginas: tabela, formulário, cards e modais.

style.css

Pequenas personalizações visuais. O layout principal vem do Bootstrap.

JavaScript

Onde implementaremos as chamadas fetch e a atualização dinâmica da tela.

Isso é MVC?



- Não é necessário chamar de MVC “puro” neste momento.
- O conceito importante é separar responsabilidades.
- A tela não deve conter toda a lógica de comunicação com a API.
- O arquivo api.js concentra as chamadas HTTP e facilita manutenção.

Responsabilidade de cada arquivo JS

A divisão reduz repetição e ajuda o entendimento da aplicação por partes.

api.js

Funções que conversam com o backend:
- listarAlunosApi
- buscarAlunoApi
- cadastrarAlunoApi
- atualizarAlunoApi
- inativarAlunoApi

alunos-lista.js

Controla a página de listagem:
- carregar tabela
- aplicar filtro
- abrir modais
- atualizar cards

aluno-formulario.js

Controla a página de cadastro/edição:
- buscar por RA
- montar objeto
- salvar
- preencher formulário

Ideia-chave: cada arquivo deve ter uma responsabilidade clara.

O que é fetch?

fetch é a função do navegador usada para fazer requisições HTTP.

```
async function carregarAlunos() {
  const resposta = await
fetch("http://localhost:3000/alunos");
  const alunos = await resposta.json();

  console.log(alunos);
}
```

- `fetch(...)` envia uma requisição para a API.
- `await` espera a resposta chegar.
- `resposta.json()` converte o retorno para objeto/array JS.
- A página usa esses dados para atualizar o HTML.

Atenção
fetch não “mostra dados na tela” sozinho. Ele apenas busca ou envia dados. Quem atualiza a tela é o nosso JavaScript.

GET: listando alunos

Primeiro passo: buscar a lista de alunos na API.

```
async function listarAlunosApi(situacao = "") {
  let url = API_BASE_URL + "/alunos";

  if (situacao !== "") {
    url += "?situacao=" + situacao;
  }

  const resposta = await fetch(url);
  const dados = await resposta.json();
  return dados;
}
```

Sem filtro

GET /alunos
Retorna todos os alunos cadastrados.

Com filtro

GET /alunos?situacao=Ativo
Usa req.query.situacao no backend.

Resultado

A função retorna um array de alunos para a página renderizar.

Da API para a tabela HTML

Depois de receber o JSON, precisamos transformar dados em linhas da tabela.

```
function renderizarTabela(alunos) {
  const tabela = document.getElementById("tabelaAlunos");
  tabela.innerHTML = "";

  alunos.forEach((aluno) => {
    tabela.innerHTML += "<tr>" +
      "<td>" + aluno.ra + "</td>" +
      "<td>" + aluno.nome + "</td>" +
      "<td>" + aluno.curso + "</td>" +
      "<td>" + aluno.situacao + "</td>" +
      "</tr>";
  });
}
```

- Seleciona o `tbody` pelo ID.
- Limpa a tabela antes de preencher.
- Percorre o array com `forEach`.
- Cria uma linha HTML para cada aluno.
- Insere botões de ação quando necessário.

Cuidado

Se não limpar `innerHTML`, a tabela duplica os dados a cada recarregamento.

Filtrando por situação

O filtro da tela usa query parameter no backend.

```
async function aplicarFiltroSituacao() {
  const select = document.getElementById("filtroSituacao");
  const situacao = select.value;

  const alunos = await listarAlunosApi(situacao);
  renderizarTabela(alunos);
  atualizarCardsResumo(alunos);
}
```



GET por RA: buscando um aluno específico

A rota /alunos/:ra identifica um recurso específico.

```
async function buscarAlunoApi(ra) {
  const resposta = await fetch(API_BASE_URL + "/alunos/"
+ ra);
  const dados = await resposta.json();

  return dados;
}
```

Frontend chama

GET http://localhost:3000/alunos/1

Backend recebe

req.params.ra → "1"

Resposta

Aluno encontrado ou mensagem de erro 404

Uso na aula

Essa função será usada tanto para abrir detalhes no modal quanto para preencher o formulário de edição.

POST: cadastrando aluno pelo formulário

O formulário cria um objeto JavaScript e envia JSON para a API.

```
async function cadastrarAlunoApi(aluno) {
  const resposta = await fetch(API_BASE_URL + "/alunos", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(aluno)
  });

  return await resposta.json();
}
```

- `method: "POST"` indica cadastro.
- `headers` informa que o corpo está em JSON.
- `JSON.stringify` transforma objeto JS em texto JSON.
- O backend lê os dados em `req.body`.
- Resposta esperada: `201 Created`.

Ponto de conexão

POST /alunos no frontend → app.post("/alunos") no backend.

PUT: atualizando aluno existente

Para atualizar, usamos RA na rota e os novos dados no body.

```
async function atualizarAlunoApi(ra, aluno) {
  const resposta = await fetch(API_BASE_URL + "/alunos/" + ra, {
    method: "PUT",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(aluno)
  });

  return await resposta.json();
}
```

Na prática:

params

Qual aluno será atualizado?
/alunos/1

body

Quais dados novos serão enviados?
nome, email, curso, semestre, situação

modo edição

A página controla se deve cadastrar ou atualizar.

DELETE lógico: inativando aluno

A rota DELETE não remove o aluno: altera a situação para Inativo.

```
async function inativarAlunoApi(ra) {
  const resposta = await fetch(API_BASE_URL + "/alunos/"
+ ra, {
    method: "DELETE"
  });

  return await resposta.json();
}
```

Por que “lógico”?

- O registro continua no array/banco.
- A situação muda para `Inativo`.
- Preserva histórico e evita perda de dados.
- A tela pode filtrar ativos/inativos.

Conexão com o projeto final

No sistema de demandas, a “exclusão” também será cancelamento lógico: o registro permanece, mas muda de status.

Modal Bootstrap: detalhes e confirmação

O modal permite abrir uma janela sobre a tela sem trocar de página.

```
async function abrirModalDetalhes(ra) {
  const aluno = await buscarAlunoApi(ra);

  document.getElementById("modalNome").innerText = aluno.nome;
  document.getElementById("modalEmail").innerText = aluno.email;

  const modal = new bootstrap.Modal(
    document.getElementById("modalDetalhesAluno")
  );

  modal.show();
}
```

Modal de detalhes

Busca um aluno pelo RA e preenche dados na janela.

Modal de inativação

Confirma uma ação perigosa antes de chamar DELETE.

Bootstrap.Modal

Objeto JavaScript do Bootstrap usado para abrir e fechar o modal.

Tratamento básico de erros

A resposta HTTP precisa orientar a mensagem da tela.

```
async function buscarAlunoApi(ra) {
  const resposta = await fetch(API_BASE_URL + "/alunos/" + ra);
  const dados = await resposta.json();

  if (!resposta.ok) {
    throw new Error(dados.mensagem || "Erro na requisição");
  }

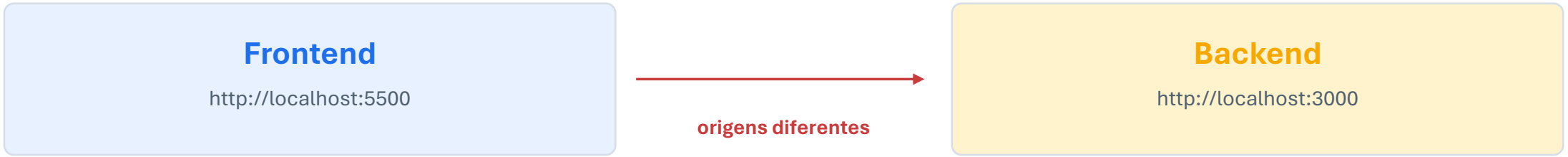
  return dados;
}
```

- `resposta.ok` é true para status 200–299.
- 404 deve virar mensagem “Aluno não encontrado”.
- 400 deve indicar dados inválidos ou incompletos.
- try/catch permite mostrar alertas amigáveis no HTML.
- Não basta olhar apenas o JSON: o status também importa.

> **Mensagem visual**
>
> Use alert Bootstrap em #areaMensagem para success, danger, warning ou info.

CORS: por que vai aparecer?

Frontend e backend rodam em origens diferentes durante o desenvolvimento.




```
npm install cors
npm install -D @types/cors

import cors from "cors";

app.use(cors());
app.use(express.json());
```

Explicação curta

CORS é uma regra de segurança do navegador. O backend precisa autorizar que a página em outra origem acesse a API.

Checkpoint 1: GET + tabela

Primeira entrega parcial da aula.

O que deve funcionar

- Abrir listar-alunos.html
- Clicar em “Carregar alunos”
- Fazer GET /alunos
- Receber JSON
- Exibir linhas na tabela
- Atualizar cards de resumo

Como testar

1. Rodar backend na porta 3000
2. Abrir frontend no navegador
3. Conferir Network/Console
4. Validar se os dados batem com a API
5. Testar filtro por situação

Critério: a tela não pode ter dados fixos. A tabela deve vir da API.

Checkpoint 2: POST, PUT e DELETE

Segunda entrega parcial: manipulação de dados pelo frontend.

Cadastrar

Formulário → objeto aluno → POST /alunos → 201 Created → mensagem de sucesso

Editar

Buscar RA → preencher formulário → PUT /alunos/:ra → tabela atualizada

Inativar

Modal confirma → DELETE /alunos/:ra → situação = Inativo

Orientação para depuração

Quando algo falhar, verificar: Console do navegador, aba Network, terminal do backend e URL chamada pelo fetch.

Erros comuns nesta aula

Antecipar problemas evita perda de tempo em laboratório.

- Backend não está rodando ou está em outra porta.
- URL do fetch escrita diferente da rota do Express.
- CORS não habilitado no backend.
- Esquecer `Content-Type: application/json` no POST/PUT.
- Esquecer `JSON.stringify(aluno)` no body.
- Tentar usar `await` fora de uma função `async`.
- IDs do HTML diferentes dos usados em `getElementById`.
- Não tratar resposta 404 ou 400 antes de atualizar a tela.

Checklist rápido

1. URL correta?
2. Método correto?
3. Body enviado?
4. CORS ativo?
5. Console mostra erro?
6. API responde no Thunder?